ANTEATER CHESS
USER MANUAL
VERSION 1.1

TEAM 21

Naveen Khan
Mia Ness
Austin Nguyen
Queency Tan
Aman Macwan
Gurvin Dhanjal

TABLE OF CONTENTS

1. COMPUTER CHESS

1.1 Usage Scenario - 4
1.2 Goals - 4
1.3 Features - 5

2. INSTALLATION

2.1 System Requirements - 8
2.2 Setup and Configuration - 8
2.3 Uninstalling - 9

3. CHESS PROGRAM FUNCTIONS AND FEATURES

3.1 Function: Move - 10
3.2 Function: Capture - 10
3.3 Function: Anteater Capture - 10
3.4 Function: Castle - 11
3.5 Function: En Passant - 11
3.6 Function: Promotion - 11
3.7 Function: Check / Checkmate / Stalemate - 12
3.8 Function: Per-Side Timer - 12
3.9 Function: Turn - 13
3.10 Function: New Game - 13
3.11 Function: End Game - 13
3.12 Function: Undo - 14
3.13 Function: Quit - 14

4. COPYRIGHT

4.1 Copyright - 15

5. ERROR AND STATUS MESSAGES

5.1 Error and Status Messages - 16

6. INDEX

6.1 Index - 17

GLOSSARY

ANT — Another name for a pawn in Anteater Chess. Moves forward one square (two if moving from its starting position) and captures diagonally. If an Ant reaches the opposite end of the board, it is promoted to another piece of the player's choice.

ANTEATER — Piece unique to Anteater Chess. Moves one square in any direction, like a King. The Anteater is only allowed to capture Pawns (Ants), with the ability to capture multiple in a single turn by chaining adjacent captures.

BISHOP — Piece that moves diagonally across the board any number of squares.

CASTLE — A special move where the King and a Rook move at the same time. When there are no pieces between the King and the chosen Rook, and neither piece has moved, the King moves two squares toward the Rook and the Rook moves to the square the King passed over.

CHECK — A state in which a player's King is under direct attack by an opposing piece. The player must resolve the check on their next move.

CHECKMATE — A check from which a King has no legal way to escape. The game ends and the attacking player wins.

CPU — The computer-controlled opponent. The CPU plays at one of three difficulty levels (Easy, Medium, Hard) selected when starting a game.

EN PASSANT — A special pawn capture. If an opposing Ant moves two squares from its starting position and lands beside one of your Ants, your Ant may capture it as if it had moved only one square — but only on the immediately following turn.

KING — Moves one square in any direction. The game ends when a King is checkmated.

KNIGHT — Piece that moves in an L-shape: two squares in one direction and one square perpendicular. The only piece with the ability to jump over other pieces.

PAWN — See ANT.

PROMOTION — When an Ant reaches the back rank of the board, it is promoted to a Queen, Anteater, Bishop, Knight, or Rook of the same color, chosen by the player from a pop-up menu.

QUEEN — Piece that can move horizontally, vertically, and diagonally any number of squares.

ROOK — Piece that moves horizontally and vertically any number of squares.

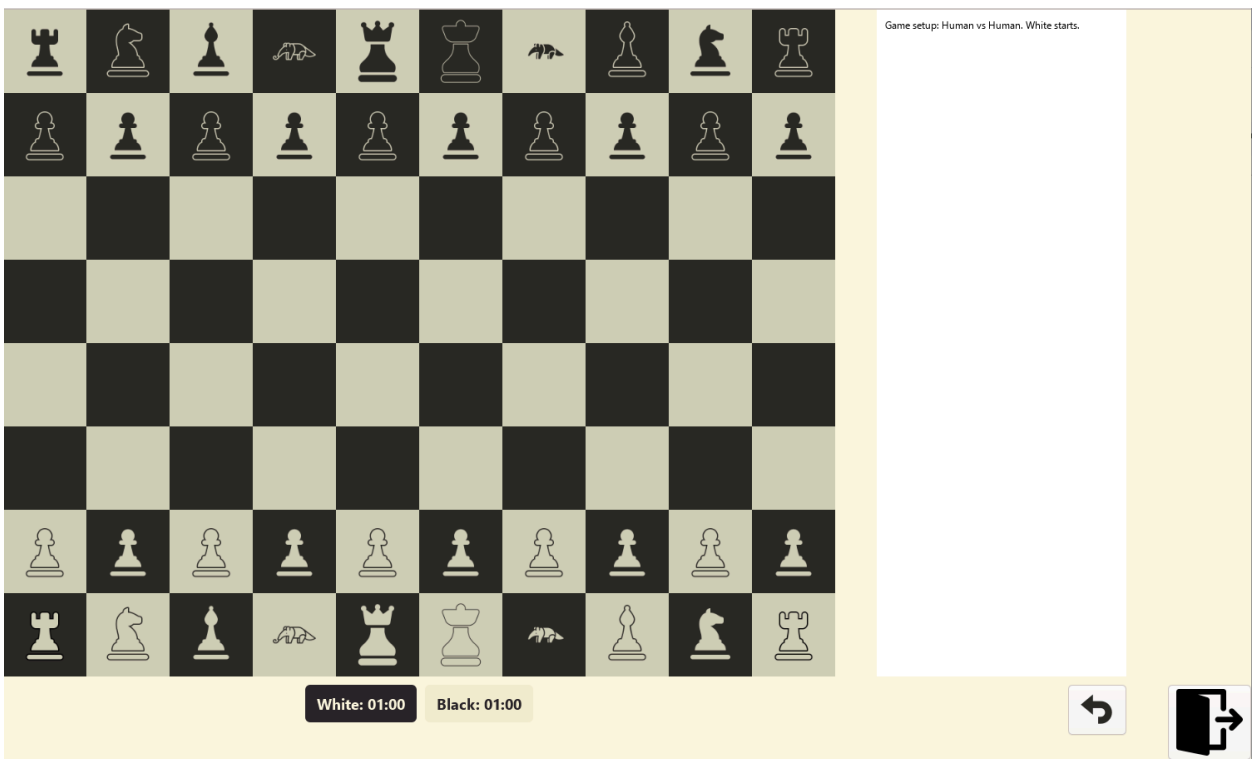
STALEMATE — A state where the player whose turn it is has no legal moves but is not in check. The game ends in a draw.

1.1 Usage Scenario

The user launches the Anteater Chess executable from the bin/ directory. The main game window opens and a Game Setup dialog appears asking the user to choose the game mode (Human vs CPU or Human vs Human). For Human vs CPU mode, the user also chooses which color to play (white or black) and which difficulty level to use (Easy, Medium, or Hard). The user selects Human vs CPU, white, and Medium, then clicks "Start Game."

The main window now shows the 8x10 chess board fully populated with pieces, a move log on the right, two countdown timers above the board (one for each side), and undo and quit buttons. White moves first; the white timer counts down from 1:00 while the white player thinks. The user clicks on an Ant at F2 and the squares F3 and F4 are highlighted in green to indicate the legal moves. The user clicks F4. The Ant moves and the entry "White: F2 -> F4" is appended to the move log. The CPU then moves an Ant from F7 to F5, and "CPU: F7 -> F5" is logged.

Later in the game, the user moves their Anteater next to a row of CPU Ants. The Anteater captures the first Ant; because another enemy Ant is orthogonally adjacent, the message "Anteater can capture again" is logged and the Anteater keeps the turn, eating the next Ant, and so on. Eventually the user delivers a checkmate. A Game Over dialog reads "Checkmate. White wins." along with the total number of moves played, and offers "New Game" or "Quit".



Reference 1: Team 21's 8x10 starting board

1.2 Goals

The goal of the program is to simulate a complete chess game with a programmed opponent. The program automates one side of the game so a human can play against it. The CPU is restricted to legal moves only and plays with some level of strategy that scales with the chosen difficulty. The game enforces a one-minute-per-turn time limit for each side, signals when a player is in check, and detects checkmate and stalemate automatically. When a piece is selected, all of its legal destination squares are highlighted, and neither the human player nor the CPU is allowed to make a move outside the rules of any piece.

1.3 Features

Anteater Chess provides a rich set of features that deliver a complete chess experience with a twist. The following features are included in Version 1.0:

Custom Anteater Piece (Reference 1.31)

- A piece unique to Anteater Chess. Moves one square in any direction (like a King) but is restricted to capturing Ants (Pawns) only and cannot deliver check.
- The Anteater can capture a chain of Ants in a single turn: after a successful capture, if another enemy Ant is orthogonally adjacent, the Anteater keeps the turn and may capture again. This continues until no adjacent enemy Ant remains.
- Captured Ants are removed from the board immediately, square by square, as the chain progresses.

8x10 Game Board (Reference 1.32)

- The game is played on an expanded 8x10 board (versus the typical 8x8) to accommodate two Anteater pieces on each side.
- Board columns are labeled A through J and rows are numbered 1 through 8 for standard algebraic notation. Coordinates are displayed in the move log as "<color>: <start> -> <end>", e.g. "White: F2 -> F4".

Game Mode Selection

- Human vs CPU: play against the built-in computer opponent.
- Human vs Human: two players take turns on the same machine, white moving first.
- Game mode is chosen from a Game Setup dialog at launch and again whenever a new game is started.

AI Computer Opponent with Difficulty Levels

- The program includes a computer-controlled opponent that plays legal, strategically informed moves.
- Three difficulty levels are available: Easy (random legal move), Medium (best immediate move based on a positional and capture-value heuristic), and Hard (alpha-beta minimax search to depth 2 over the same heuristic).
- In Human vs CPU mode the player may choose to play as White or Black; if Black is chosen, the CPU plays the first move automatically.

Move Highlighting and Legal Move Enforcement (Reference 1.33)

- When a piece is selected, every legal destination square is highlighted on the board, including squares reached by special moves such as castling and en passant.
- The program strictly enforces all chess rules — neither the human player nor the CPU can make an illegal move, and any move that would leave one's own King in check is rejected.

Per-Side Countdown Timer

- Each side has its own one-minute timer displayed above the board ("White: 00:60" / "Black: 00:60").

- The active side's timer counts down once per second and is highlighted while it is that side's turn. After a move is completed, the moving side's timer is reset to 60 seconds and the other side's timer becomes active.
- If a side's timer reaches zero, the opposing side wins on time.

Move Log

- A scrollable text log on the right side of the window records every move made during the game, including captures, castling, en passant, promotions, check announcements, checkmate, stalemate, and any extra Anteater capture turns.
- The same log is mirrored to a human-readable file named chess_game.log in the working directory.

Check, Checkmate, and Stalemate Detection

- After every move the program checks whether the opposing King is in check, checkmate, or stalemate.
- If the opposing King is in check, a "<Color> is in check." message is appended to the move log.
- Checkmate triggers an end-of-game dialog identifying the winner. Stalemate triggers the same dialog with a draw message.

Special Moves

- **Castling:** The King and a Rook may castle when neither piece has moved, no pieces occupy the squares between them, and the King would not move into, through, or out of check. Selecting the King highlights the castling destination if it is legal.
- **En Passant:** An Ant (Pawn) may capture an adjacent enemy Ant via en passant on the immediately following turn after the enemy Ant advances two squares from its starting position.
- **Pawn Promotion:** When an Ant reaches the back rank, a pop-up menu appears anchored to the promoting square. The player chooses Queen, Rook, Bishop, Knight, or Anteater and the Ant is replaced in place.

Undo Functionality

- An undo button restores the most recent move, including special moves: castling moves both pieces back, en passant restores the captured Ant beside the moving Ant, and ordinary captures replace the captured piece on its square.
- Undoing an Anteater chain capture restores every Ant consumed during that chain in a single click.

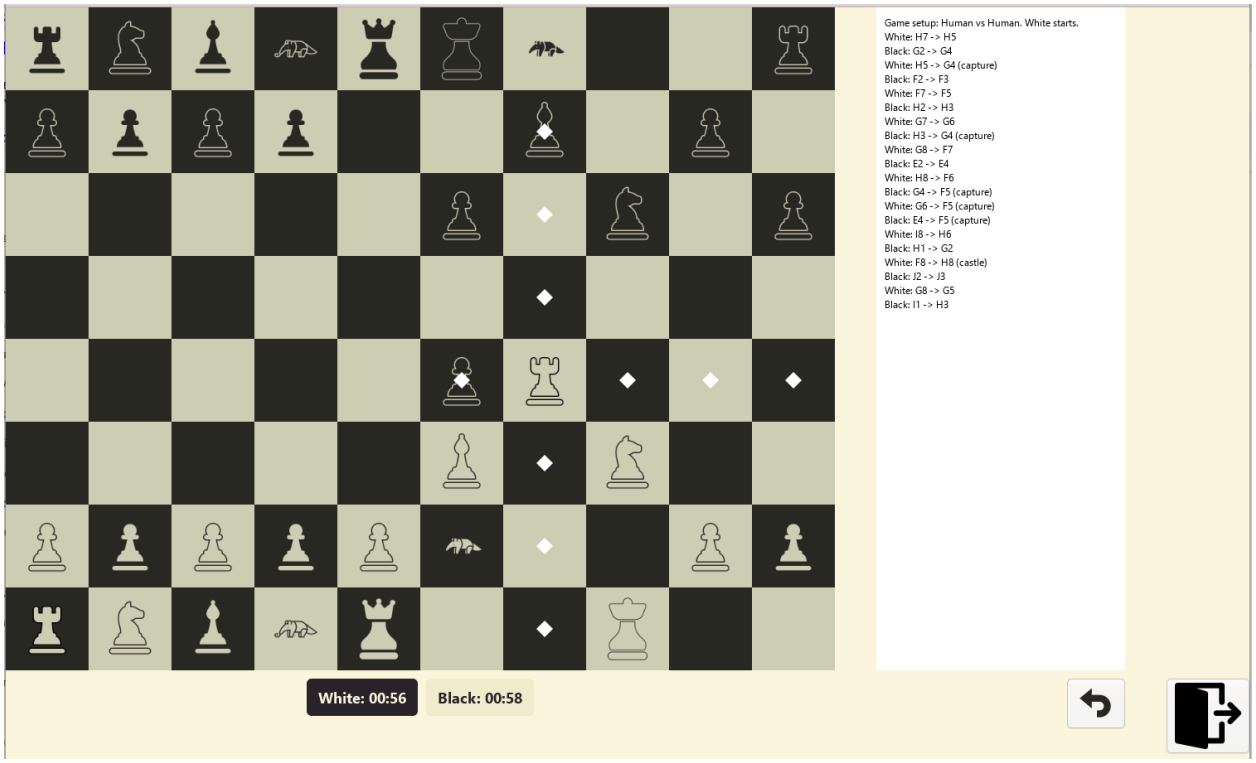
New Game and End-of-Game Summary

- At the conclusion of a game, an end-game dialog displays the result (winner or stalemate) along with the total number of moves played.
- From the end-game dialog the player may start a new game (which re-opens the Game Setup dialog) or quit the application.

Quit

- A quit button is available at all times. Clicking it opens a confirmation dialog ("Are you sure you want to quit?"), and only a confirmed Yes terminates the program. Quitting is also recorded in chess_game.log.

Reference 2: Anteater Chess Piece



Reference 3: Highlighting Moves using Rook

2.1 System Requirements

Before installing and running the program, please make sure your system meets the following requirements:

- Linux operating system (tested on UCI EECS lab machines)
- GCC compiler with C99 support
- GNU Make
- GTK 4 development libraries and pkg-config
- Cairo and GDK-Pixbuf (typically installed alongside GTK 4)
- Ability to use a Linux terminal

2.2 Setup and Configuration

Two distribution archives are provided. Choose whichever is appropriate for your situation:

Installing from the source package

Step 1: Extract the source archive.

```
$ gtar -xvzf Chess_V1.0_src.tar.gz
```

Step 2: Change into the extracted directory.

```
$ cd Chess_V1.0_src
```

Step 3: Build the program.

```
$ make
```

The executable is placed in bin/ as bin/chess.

Step 4: (Optional) Run the test suite.

```
$ make test
```

Step 5: Run the program.

```
$ ./bin/chess
```

Installing from the binary package

Step 1: Extract the binary archive.

```
$ gtar -xvzf Chess_V1.0.tar.gz
```

Step 2: Change into the extracted directory.

```
$ cd Chess_V1.0
```

Step 3: Run the program.

```
$ ./bin/chess
```

2.3 Uninstalling

To remove the program from your system:

Step 1: Clean compiled files.

```
$ make clean
```

Step 2: Remove the project directory.

```
$ cd .. && rm -rf Chess_V1.0_src
```

Function 1: Move

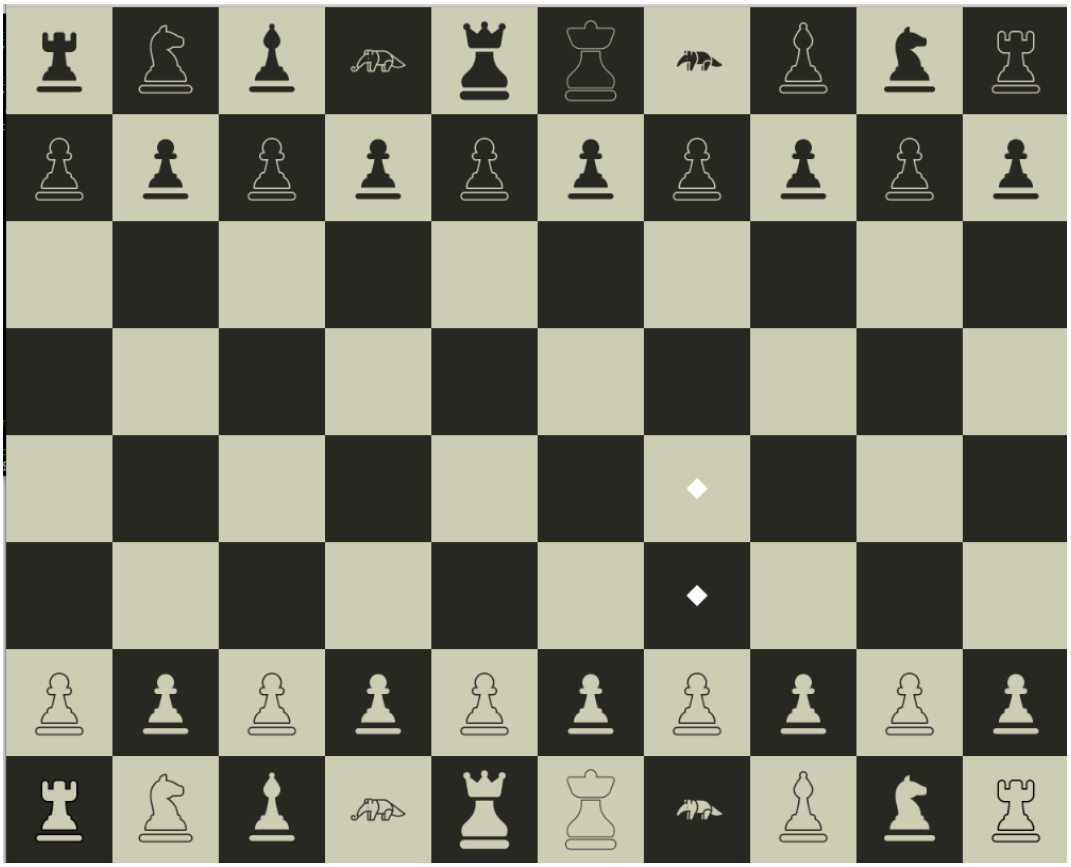
Description: Moves a piece from one square to another according to the rules of the piece.

Inputs: The user clicks on a piece, then clicks the destination square. Internally: piece color, current turn, board state, last move.

Outputs: The piece is moved on the board, the move is appended to the on-screen move log and to chess_game.log, and the GUI is redrawn.

Logic: The program verifies that the clicked piece belongs to the side to move, generates the set of legal destinations, and only allows the move if the destination square is in that set and the resulting position does not leave the moving side's King in check.

Error: If the second click is not on a legal destination, the selection is cleared and no move is made.



Function 2: Capture

Description: Captures an opposing piece by moving onto its square.

Inputs: Move function output, target piece, piece color.

Outputs: The captured piece is removed from the board and the moving piece is placed on the captured piece's square; the action is logged as "<Color>: <start> -> <end> (capture)" in both the on-screen log and chess_game.log.

Logic: Performed as part of executeMove when the destination square holds an opposing piece. Pawn diagonal captures and Anteater captures use specialized capture routines.

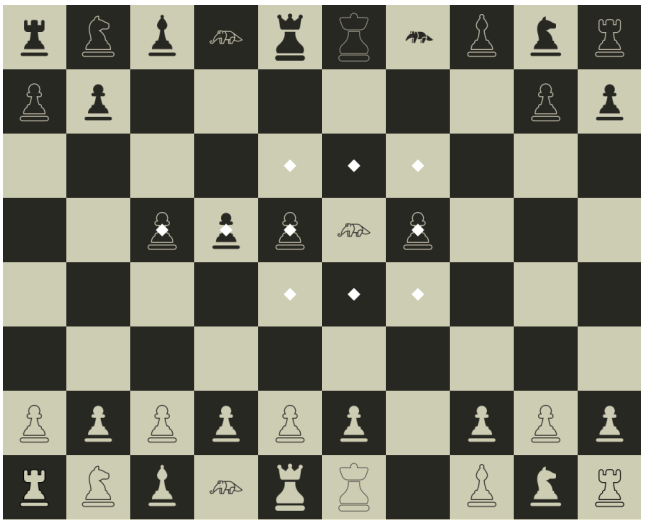
Function 3: Anteater Capture

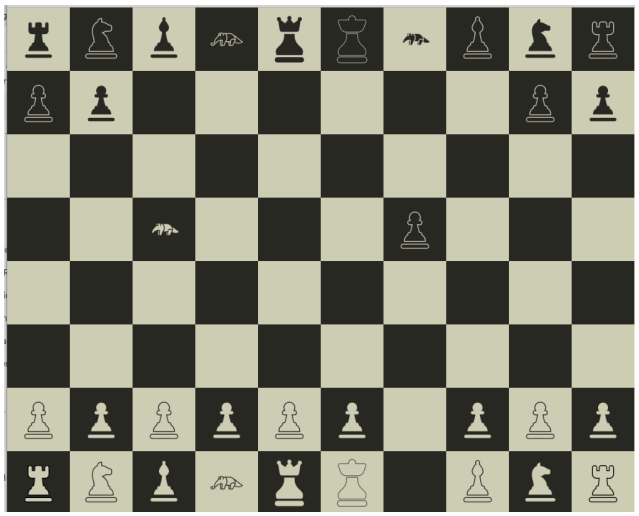
Description: The Anteater eats a chain of orthogonally adjacent enemy Ants in a single turn.

Inputs: Anteater move, board state, piece color.

Outputs: Each captured Ant is removed in turn; "Anteater can capture again" is logged after each link in the chain. The turn ends only when no enemy Ant is orthogonally adjacent to the Anteater.

Logic: After the initial capture, the program calls anteaterHasAdjacentPawn. If true, the Anteater is re-selected, fresh legal moves are generated, and the user clicks the next Ant to capture; otherwise control passes to the other side.





Function 4: Castle

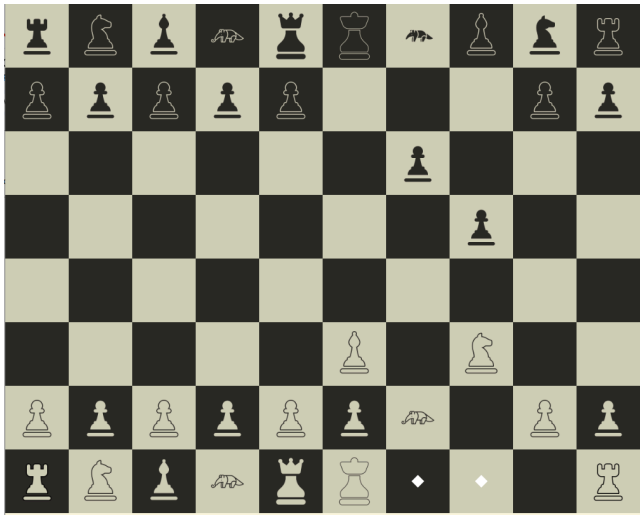
Description: Switches the position of the King and a chosen Rook.

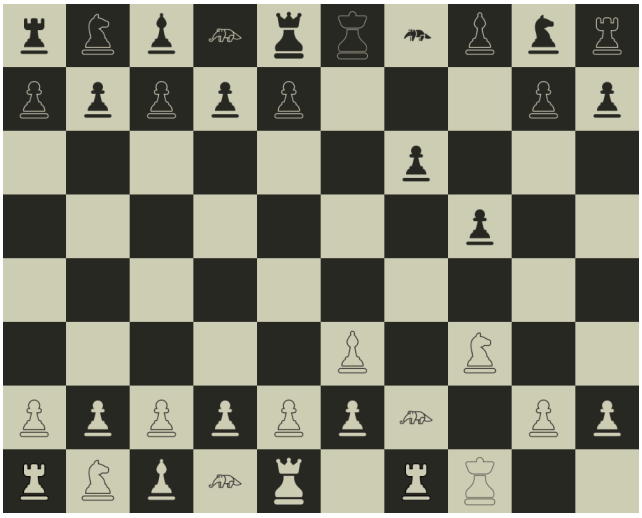
Inputs: King position, Rook position, piece color.

Outputs: The King moves two squares toward the Rook and the Rook is placed on the square the King crossed; the move is logged as a castle in both the on-screen log and chess_game.log.

Logic: Verifies that neither the King nor the chosen Rook has moved before, that the squares between them are empty, and that the King is not in check, would not pass through check, and would not land in check.

Error: If any of these conditions fails, the destination square is not highlighted and the move cannot be made.





Function 5: En Passant

Description: An Ant captures an adjacent enemy Ant that just advanced two squares from its starting position.

Inputs: Move function output, position of the moving Ant, position of the enemy Ant, last move.

Outputs: The enemy Ant is removed and the moving Ant is placed on the diagonal square behind it; logEnPassant is called and the action appears in both logs.

Logic: The capture is only legal on the turn immediately after the enemy Ant's two-square advance, as recorded in the lastMove field of the board state.

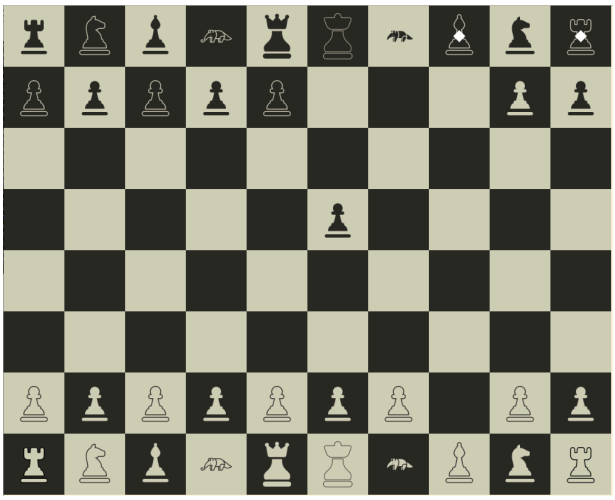
Function 6: Promotion

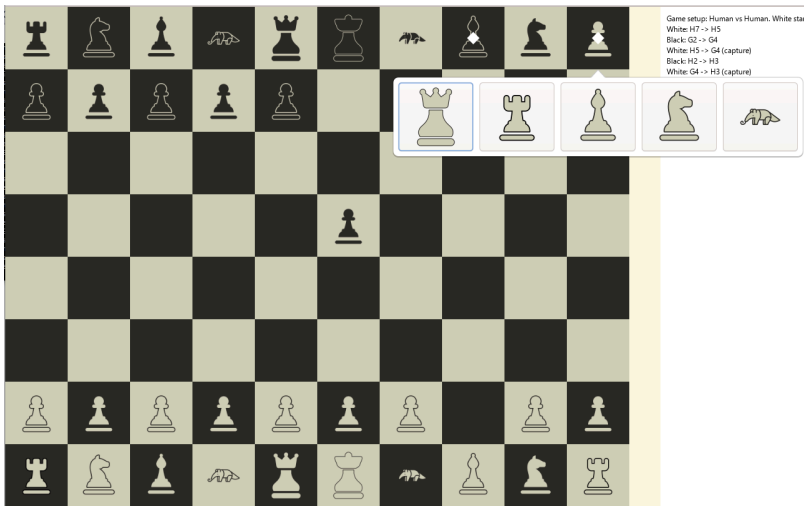
Description: Promotes an Ant to a Queen, Rook, Bishop, Knight, or Anteater when it reaches the back rank.

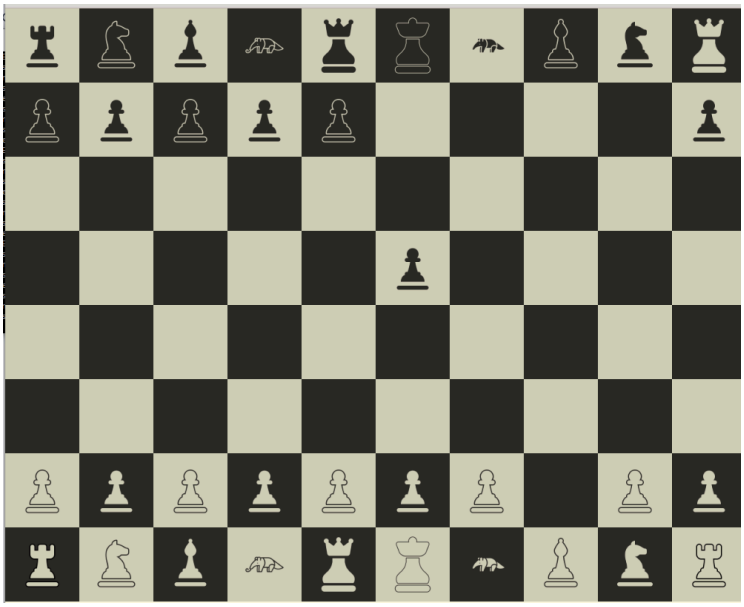
Inputs: Position of the promoting Ant, the player's chosen rank from the pop-up.

Outputs: The Ant is replaced in place by a piece of the chosen rank in the same color; the promotion is logged via logPromotion.

Logic: When executeMove leaves an Ant on row 0 (white) or row 7 (black), the program opens a pop-up anchored to the promoting square. The promoted-to piece's rules immediately become active.







Function 7: Check / Checkmate / Stalemate

Description: Detects whether a King is in check, checkmate, or stalemate after each move.

Inputs: Board state, color of the side to move next.

Outputs: If the next side is in check but has at least one legal move, a "<Color> is in check." message is logged. If the next side is in check with no legal move, an end-game dialog announces checkmate and the moving side wins. If the next side has no legal moves and is not in check, an end-game dialog announces stalemate.

Logic: Called at the end of each completed turn. The program scans every piece of the side to move for any legal move; the absence of a legal move combined with the current check status determines the result.

Function 8: Per-Side Timer

Description: Per-side one-minute countdown that tracks how long each player has remaining for the current turn.

Inputs: Current turn, move completion events.

Outputs: Two timer labels are displayed above the board ("White: 00:60" and "Black: 00:60"). The active side's label is visually highlighted.

Logic: A 1-second tick decrements the active side's seconds. When a move completes, the moving side's timer is reset to 60 seconds and the other side's timer becomes active. If a timer reaches zero, the opposing side is declared the winner.

Function 9: Turn

Description: Tracks whose turn it is to move.

Inputs: Total moves made so far.

Outputs: Determines who can make the next click on the board. White moves on even move counts, Black on odd.

Logic: Click events on the board are ignored unless the clicked piece's color matches the side to move. In Human vs CPU mode, the board is also unresponsive while it is the CPU's turn.

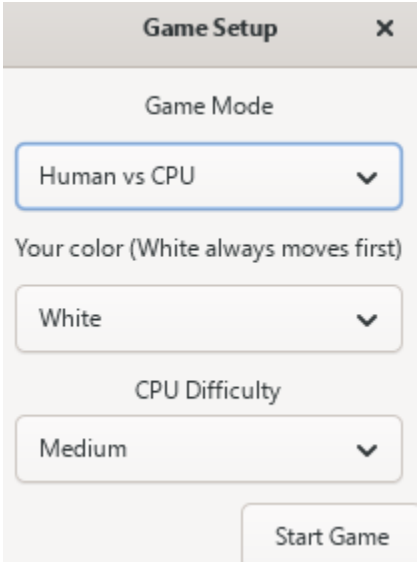
Function 10: New Game

Description: Starts a new game from the standard starting position.

Inputs: Game mode (Human vs CPU or Human vs Human), human player color, CPU difficulty (CPU mode only).

Outputs: The board is re-initialized to its default layout, the move log is cleared, both timers are reset to 60 seconds, and the move counter is reset to zero. White moves first.

Logic: Triggered at program launch and from the "New Game" button on the end-game dialog. If the user is Black in Human vs CPU mode, the CPU plays the first move automatically after the dialog closes.



Function 11: End Game

Description: Displays an end-game dialog when the game ends.

Inputs: Outcome (checkmate, stalemate, or time-out), winning color (if applicable), total moves made.

Outputs: A modal dialog with a result message and the total move count, plus "New Game" and "Quit" buttons.

Logic: Invoked from finishGameIfNeeded and from the timer tick when a side runs out of time.

Function 12: Undo

Description: Reverses the most recent completed move.

Inputs: The undo stack, current board state.

Outputs: The board is restored to its state before the last move; if a piece was captured, it is restored on its original square; if a castling or en passant move was made, both affected pieces are restored. The on-screen log is updated.

Logic: Each completed move pushes an Undo_Record. Undo pops the top record and applies its inverse, including the special-move fields (movedRook, rookMove, movedPieceMovedBefore, rookMovedBefore) when the record represents a castle.

Function 13: Quit

Description: Exits the program.

Inputs: User clicks the quit button.

Outputs: A confirmation dialog appears. On Yes, the application terminates and "<Color> quits" is recorded in chess_game.log; on No, the dialog is dismissed and play continues.

COPYRIGHT

© 2026 Team 21 — EECS 22L, University of California, Irvine. All rights reserved.

This document and the accompanying Anteater Chess software are produced as coursework for EECS 22L, Spring 2026. No part of this document or its associated source code may be reproduced, distributed, or transmitted in any form without the prior written permission of the authors, except for academic evaluation by course instructors and teaching assistants.

Anteater Chess is an original adaptation of the traditional chess game, developed exclusively for educational purposes at the University of California, Irvine. The Anteater piece and associated rules are defined by the EECS 22L course specification authored by Prof. Rainer Doemer.

ERROR AND STATUS MESSAGES

The following messages may appear in the on-screen move log or in pop-up dialogs during gameplay. Most are status updates rather than errors; the program does not generate textual errors for illegal moves — illegal destinations simply are not highlighted and clicks on them clear the selection.

Game setup (CPU): *"Game setup: You are <Color>. CPU is <Color> (<Difficulty> difficulty). White starts."*

Game setup (Human): *"Game setup: Human vs Human. White starts."*

Move: *"<Color>: <Start> -> <End>"*

Capture: *"<Color>: <Start> -> <End> (capture)"*

Castle: *"<Color>: <Start> -> <End> (castle)"*

CPU move: *"CPU: <Start> -> <End>"*

Anteater chain: *"Anteater can capture again"*

Check: *"<Color> is in check."*

Checkmate: *"Checkmate. <Color> wins."*

Stalemate: *"Stalemate."*

Time-out (white): *"White ran out of time. Black wins."*

Time-out (black): *"Black ran out of time. White wins."*

Quit prompt: *"Are you sure you want to quit?"*

INDEX

A

Ant, 4, 5, 10, 11
Anteater, 4, 5, 10
Anteater capture, 4, 5, 10

B

Bishop, 4
Board, 8x10, 5, 6, 7
Build (make), 8

C

Capture, 10
Castle, 5, 11
Check, 5, 12
Checkmate, 5, 12
CPU difficulty, 5, 8
CPU mode, 5

D

Difficulty (Easy/Medium/Hard), 5, 8

E

End game dialog, 6, 13
En passant, 5, 11
Errors and messages, 16

G

Game mode dialog, 5, 13
Game Over dialog, 13
GTK 4, 8

H

Human vs CPU, 5
Human vs Human, 5

I

Installation, 8

K

King, 4
Knight, 4

L

Legal move highlighting, 5, 7
Linux, 8
Log file (chess_game.log), 5, 16

M

Move (function), 10
Move log, 5

N

New Game, 6, 13

P

Pawn (Ant), 4
Pop-up (promotion), 5, 11
Promotion, 5, 11

Q

Queen, 4
Quit, 6, 14

R

Rook, 4

S

Stalemate, 5, 12
System requirements, 8

T

Time-out, 5, 12
Timer (per-side), 5, 12
Turn, 13

U

Undo, 6, 14
Uninstalling, 9
Usage scenario, 4

W

White moves first, 5, 13